



Specification Patterns Nâng Cao

PLAYBOOK SDD + ADD · CHƯƠNG 7

"Spec as Code" — Khi đặc tả trở thành lớp kiểm soát chất lượng sống

Giới thiệu chương

Tư duy "Spec as Code"

Chương 6 đã cho bạn workflow. Chương 7 tiến thêm một bước: biến spec từ "tài liệu con người đọc" thành "artifact máy có thể xử lý". Đây là bước nhảy vọt trong tư duy — *Spec as Code* — nơi các file đặc tả trở thành lớp enforcement tự động, không còn phụ thuộc vào việc có ai đó nhớ check hay không.

Sáu patterns trong chương này không phải kỹ thuật cô lập — chúng liên kết với nhau thành một hệ thống hoàn chỉnh.

Spec as Code — 3 Nguyên tắc Cốt lõi



Spec được lưu trong Git cùng code — không phải trong Confluence hay Word



Spec có thể được đọc và xử lý tự động bởi scripts và AI agents



Violations của spec được phát hiện tự động (CI/CD) — không phải bằng code review thủ công

Nội dung Chương 7

1	7.1 — Constitution-Driven Development Giải phẫu · Template · Enforcement · RFC
2	7.2 — Clarification-First Planning Tại sao AI không hỏi · Trigger Prompt · Categories
3	7.3 — Consistency Analysis Gate Spec-Code Drift · Cross-Artifact · Sync-back
4	7.4 — Parallel Implementation Exploration Khi nào dùng · Prompt · Decision Framework
5	7.5 — Specification Scale Management Spec Hierarchy · Git Strategy · Spec Debt
6	7.6 — Case Study — EcoShop Greenfield Cart Service · Payment · Lessons Learned

SECTION 7.1

Constitution-Driven Development

Một tập quy tắc bất biến mà mọi spec, mọi design, mọi dòng code đều phải tuân thủ — bất kể feature nào đang được build

Giải phẫu một Constitution (Hiến pháp)

Mọi hệ thống pháp lý văn minh đều có một bản hiến pháp — tập hợp các nguyên tắc không thể bị override bởi bất kỳ luật cụ thể nào. Constitution-Driven Development áp dụng triết lý đó cho phần mềm. Điểm khác biệt: Constitution là input cho AI tự kiểm tra chính nó.

Lớp	Tên	Đặc điểm	Ví dụ vi phạm
Layer 1	Hard Rules	Không bao giờ được vi phạm, CI fail ngay	Lưu secret trong code, dùng MD5 cho password
Layer 2	Architectural Constraints	Cần approved exception để bypass	Service A gọi trực tiếp DB của Service B
Layer 3	Engineering Standards	Có thể override với documented reason	Test coverage < 80% cho module cụ thể

 Key Insight

Constitution không chỉ là checklist cho con người đọc. Trong mô hình Spec as Code, Constitution là input cho AI tự kiểm tra chính nó. Trước khi AI submit bất kỳ output nào, nó chạy lại Constitution như một test suite và tự báo cáo violations.

LAYER 1 — Hard Rules

CONSTITUTION.md — Layer 1: Hard Rules (không bao giờ vi phạm)

PROJECT CONSTITUTION — [Project Name]

Version: 1.0.0 | Owner: @tech-lead

Status: LOCKED — chỉ thay đổi qua RFC process

Áp dụng cho: mọi AI agent, mọi developer, mọi PR

SEC-01: Bảo mật thông tin

THE system SHALL NOT lưu bất kỳ secret nào dưới dạng plaintext trong source code, config files, hoặc logs.

Áp dụng cho: API keys, passwords, tokens, PII, NHNN data.

Enforcement: git-secrets pre-commit hook (tự động).

SEC-02: Authentication bắt buộc

THE system SHALL yêu cầu xác thực cho mọi endpoint thay đổi dữ liệu (POST, PUT, PATCH, DELETE).

Ngoại lệ: public endpoints phải được document rõ lý do.

SEC-03: Input validation

THE system SHALL validate và sanitize tất cả user input trước khi xử lý hoặc lưu vào database.

Không có raw SQL query với user input không được parameterize.

DATA-01: Không xóa dữ liệu vĩnh viễn

THE system SHALL dùng soft-delete (deleted_at) thay vì hard-delete cho mọi entity business-critical.

Hard-delete chỉ được phép cho: logs > 90 ngày, temp files.

LAYER 2 & 3 — Arch & Engineering

LAYER 2: Architectural Constraints

ARCH-01: Service boundary

Services SHALL giao tiếp qua API contracts (REST/gRPC/events). Direct DB access từ service khác là PROHIBITED.

Exception process: RFC trong .sdd/rfcs/
+ tech lead sign-off.

ARCH-02: Event-driven cho async operations

Operations > 2 giây SHALL được xử lý asynchronously qua message queue (Kafka/RabbitMQ). Sync HTTP call với timeout > 2s là architectural violation.

ARCH-03: Idempotency

Mọi mutating API endpoint SHALL có idempotency mechanism.

LAYER 3: Engineering Standards

ENG-01: Test coverage

Minimum test coverage: 80% cho business logic.
Exception: proof-of-concept branches
(cần xóa trước merge main).

ENG-02: Documentation

Mọi public API endpoint SHALL có OpenAPI documentation. Mọi business rule SHALL có EARS tag trong code comments.

ENG-03: Error handling

THE system SHALL không expose internal error details ra client.

Error response format: {error_code, message, request_id}. Stack trace SHALL chỉ xuất hiện trong server logs.

ENG-04: Dependency

Third-party library SHALL được pin version cụ thể. Major version update cần security review.

AI tự kiểm tra Constitution

Format Báo cáo Self-Check

```
FORMAT BÁO CÁO SELF-CHECK

=== CONSTITUTION SELF-CHECK REPORT ===
Layer 1 (Hard Rules): ✓ PASS / ✗ VIOLATION: [details]
Layer 2 (Arch):       ✓ PASS / ⚠ EXCEPTION NEEDED: [details]
Layer 3 (Standards):  ✓ PASS / ⚠ DEVIATION: [reason]
=====
```

AI AGENT SELF-CHECK PROTOCOL — Checklist

- CHECKLIST SEC:
- ☐ Không có hardcoded secrets
([grep](#): password=, key=, token=)
 - ☐ Mọi endpoint mutating có auth middleware
 - ☐ Input validation present trước DB operations

- CHECKLIST ARCH:
- ☐ Không có cross-service DB access
 - ☐ Async operations > [2s](#) dùng queue
 - ☐ Mutating endpoints có idempotency

- CHECKLIST ENG:
- ☐ Unit tests cover happy path + error cases
 - ☐ EARS tags trong code comments
 - ☐ Error responses không chứa stack trace

Nếu vi phạm phát hiện:

AI SHALL báo cáo:

"[\[CONSTITUTION VIOLATION\]](#) Rule: {ID}

File: {file}, Line: {n}.

Action taken: {description}"

AI SHALL KHÔNG submit code vi phạm Layer 1.

AI SHALL hỏi human approval cho Layer 2 violations.

Hiến pháp tự thực thi — 4 Cấp độ

Mức độ automation tăng dần theo độ trưởng thành của đội nhóm. Mỗi cấp bổ sung thêm một lớp bảo vệ:

Cấp độ	Mechanism	Khi chạy	Ví dụ tool
Cấp 1 — Instant	IDE lint rules, pre-commit hooks	Khi save/commit	ESLint, git-secrets, ruff
Cấp 2 — PR Gate	CI pipeline checks	Khi tạo PR	GitHub Actions, SonarQube
Cấp 3 — AI Self-Check	AI đọc Constitution trước submit	Trước mọi AI output	Prompt engineering + tools
Cấp 4 — Runtime	Policy enforcement (OPA/Rego)	Runtime production	Open Policy Agent

 Điểm Khác Biệt

Cấp 3 (AI Self-Check) là điểm khác biệt của Constitution-Driven Development so với coding standards thông thường. AI không chỉ nhận lệnh implement — nó tự kiểm tra output của chính mình trước khi trả về.

AI Self-Check — Output Thực tế

```
=== CONSTITUTION SELF-CHECK REPORT ===
```

```
Layer 1: ✅ PASS
```

- SEC-01: No secrets found in code
- SEC-02: Auth middleware present on POST /reviews
- SEC-03: Input validation in ReviewService.create()
- DATA-01: Using soft-delete (deleted_at)

```
Layer 2: ✅ PASS
```

- ARCH-01: No cross-service DB access
- ARCH-02: AggregateUpdate uses Kafka (async)
- ARCH-03: POST /reviews has idempotency-key support

```
Layer 3: ⚠️ DEVIATION on ENG-01
```

- BlacklistService coverage: 71% (below 80%)
- Reason: Blacklist patterns hard to unit test
- Mitigation: Integration tests cover main cases

```
=====
```

QUY TẮC

Format báo cáo self-check **LUÔN hiển thị**, kể cả khi pass tất cả. Đây là audit trail quan trọng – chứng minh rằng AI đã thực sự kiểm tra.

Constitution as CI/CD Gate

.github/workflows/constitution-check.yml

```
name: Constitution Compliance Check
on: [pull_request]

jobs:
  constitution-layer1:
    name: "Layer 1: Hard Rules (blocking)"
    steps:
      - name: Scan for hardcoded secrets
        uses: gitleaks/gitleaks-action@v2 # → Fail if secret found

      - name: Check for raw SQL with user input
        run: |
          if grep -rn "f\"SELECT|f'SELECT" src/; then
            echo "❌ CONSTITUTION VIOLATION: SEC-03"
            exit 1
          fi
          echo "✅ SEC-03: No raw SQL violations"

  constitution-layer2:
    name: "Layer 2: Arch Constraints (blocking)"
    steps:
      - name: Check cross-service imports
        run: python scripts/check_service_boundaries.py

  constitution-layer3:
    name: "Layer 3: Standards (non-blocking)"
    continue-on-error: true # Non-blocking!
    steps:
      - name: Check test coverage
        run: pytest --cov=src --cov-fail-under=80
```

Sửa đổi Hiến pháp qua RFC

Hiến pháp mạnh vì khó thay đổi — nhưng không phải không thể. RFC (Request for Comment) đảm bảo mọi thay đổi đều được thảo luận kỹ và có sự đồng thuận:

.sdd/rfcs/RFC-001-allow-direct-db-for-reporting.md

RFC Number: 001

Status: PROPOSED → DISCUSSED → ACCEPTED/REJECTED

Motivation

Reporting service cần join data từ nhiều service tables.

Hiện tại ARCH-01 prohibit cross-service DB access.

API aggregation quá chậm cho real-time reports (> 5s).

Proposed Change

Tạo [exception cho reporting-service](#): được phép dùng READ-ONLY

replica DB access cho reporting queries.

Không được dùng cho write operations.

Risk Assessment

- Service coupling tăng: HIGH risk → [mitigation](#): separate replica
- [Schema change impact](#): MED risk → [mitigation](#): versioned views

Decision

ACCEPTED — [2025-01-25](#)

[Votes](#): 4 approve, 1 reject (noted: @dev-b concerns on coupling)

[Constitution updated](#): ARCH-01 v1.1 (exception documented)

Bài tập — Constitution Design

Bài tập 7.1.A — ★★

Lấy dự án bạn đang làm (hoặc dự án open source bạn quen thuộc). Viết Constitution cho dự án đó. Phân loại mỗi rule vào đúng Layer. Sau đó: đưa Constitution cho Cline và yêu cầu nó "self-check" một đoạn code có sẵn. Báo cáo có trùng khớp không?

Bài tập 7.1.B — ★★★

Implement ít nhất 2 automated checks từ Constitution của bạn: 1 rule từ Layer 1 (blocking) và 1 rule từ Layer 3 (non-blocking). Tích hợp vào pre-commit hook hoặc GitHub Actions. Chạy thử với code vi phạm để verify enforcement hoạt động.

SECTION 7.2

Clarification-First Planning

Bắt buộc AI phải liệt kê những gì nó không chắc chắn trước khi bắt đầu bất kỳ công việc thực chất nào



Tại sao AI không tự hỏi?

AI không bao giờ nói "Tôi không hiểu" nếu không được yêu cầu. Nó sẽ chọn một cách diễn giải và tiến thẳng vào việc — với confidence cao và accuracy thấp. Hiểu nguyên nhân giúp bạn thiết kế đúng giải pháp:

Nguyên nhân	Biểu hiện	Hậu quả
Training bias toward completion	Luôn có output, kể cả khi input mơ hồ	Code dài nhưng sai requirement
Không có incentive để hỏi	Hỏi ít được reward hơn là làm nhiều	Assumption ẩn tích lũy
Context window không đặt câu hỏi	Khi đọc spec, AI không "biết mình không biết gì"	False confidence trong output

"Clarification-First Planning là một pattern đơn giản nhưng hiệu quả cao: bắt buộc AI phải liệt kê những gì nó không chắc chắn trước khi bắt đầu. Pattern này không làm chậm dự án — nó ngăn những đợt rework tốn kém hơn nhiều sau đó."

Template Chuẩn

CLARIFICATION TRIGGER PROMPT — Dán vào đầu mọi session với spec mới

Đọc SPEC.md bên dưới. TRƯỚC KHI lập bất kỳ kế hoạch hoặc viết bất kỳ code nào, hãy hoàn thành bước Clarification sau:

1. Liệt kê ít nhất 3 điểm chưa rõ ràng về:

- a) Logic nghiệp vụ (business rules không được specify rõ)
- b) Ràng buộc kỹ thuật (technical constraints còn thiếu)
- c) Edge cases (tình huống không thường xảy ra nhưng quan trọng)

2. Với mỗi điểm chưa rõ, nêu:

- Điều bạn KHÔNG chắc chắn là gì?
- Nếu bạn phải assume, bạn sẽ assume cái gì? (explicit assumption)
- Assumption này ảnh hưởng thế nào đến implementation?

3. Đặt câu hỏi cụ thể cho mỗi điểm, đủ để tôi có thể trả lời với Yes/No hoặc một giá trị cụ thể.

4. DỪNG LẠI sau khi liệt kê xong. Chờ tôi trả lời.

Không bắt đầu plan hoặc code cho đến khi có câu trả lời.

--- SPEC BẮT ĐẦU ---

[Paste SPEC.md]

--- SPEC KẾT THÚC ---

Clarification Output thực tế

AI Clarification Output — CSV Import Spec

Điểm chưa rõ — Business Logic

1. Duplicate SKU handling:

Không rõ: Spec nói "upsert" nhưng không define conflict fields.

Assume: Chỉ update price và stock, giữ name và description.

Ảnh hưởng: Nếu sai → existing product names bị overwrite.

Phân loại câu hỏi thường gặp

Sau nhiều lần chạy Clarification pattern, AI thường gặp cùng một nhóm điểm chưa rõ. Hiểu những pattern này giúp bạn chủ động làm rõ trong spec trước khi AI hỏi:

Loại câu hỏi	Ví dụ điển hình	Thường thiếu trong spec
Conflict resolution	"Nếu X và Y cùng xảy ra, ưu tiên cái nào?"	Priority ordering
Scope boundary	"Feature này xử lý trường hợp Z không?"	Out of scope items
Quantitative threshold	"Bao nhiêu là nhiều? Nhanh là bao nhanh?"	Numeric constraints
Error behavior	"Khi lỗi X, hệ thống làm gì? Retry không?"	Unwanted patterns thiếu
State transition	"Sau bước A, có thể đến bước C không?"	State diagram thiếu
Ownership	"Ai trigger cái này? User hay System?"	Actor không rõ
Technology choice	"Dùng library X hay Y?"	Tech stack constraint thiếu

Tự kiểm tra trước khi đưa spec cho AI

Thay vì đợi AI hỏi, dùng checklist này để chủ động phát hiện điểm mơ hồ:

- Mọi "nếu X thì Y" đã được specify — không có else undefined
- Mọi số: timeout, limit, threshold đã có giá trị cụ thể
- Duplicate/conflict scenarios đã được xử lý tường minh
- Mọi error path có ít nhất một Unwanted EARS pattern
- Concurrency scenarios đã được nghĩ đến (2 users cùng làm X)
- External dependencies (third-party API, DB, queue) đã được named
- State transitions đã được vẽ ra (nếu có > 3 states)

Bài tập — Clarification-First

Bài tập 7.2.A — ★★

Lấy SPEC.md từ Lab A hoặc Lab B (Chương 6). Dùng Clarification Trigger Prompt. Ghi lại: AI đặt bao nhiêu câu hỏi? Có câu nào bạn thực sự không trả lời được ngay không? Điều đó cho thấy điều gì về chất lượng spec?

Bài tập 7.2.B — ★★★

Viết một SPEC.md intentionally vague (5-6 thành phần, bỏ qua nhiều edge cases). Dùng Clarification Trigger. Sau khi nhận câu hỏi của AI, update spec để trả lời từng câu. Đo lường: spec tăng bao nhiêu dòng?



SECTION 7.3

Consistency Analysis Gate — Ngăn Spec-Code Drift

Ngăn Spec-Code Drift trước khi nó tích lũy thành debt không trả nổi

Phân loại và Nguyên nhân Drift

Mọi dự án phần mềm sống đủ lâu đều gặp Spec-Code Drift. Code tiến hóa theo sprint, theo hotfix, trong khi Spec thường nằm yên, ngày càng trở nên xa rời thực tế. Hậu quả nghiêm trọng nhất: khi dùng AI để maintain — mọi người đang làm việc dựa trên spec sai.

Loại Drift	Mô tả	Nguyên nhân thường gặp	Mức độ nguy hiểm
Silent Drift	Code thay đổi, spec không update	Hotfix vội, PR không đủ process	Cao — không ai biết
Accretion Drift	Code thêm tính năng ngoài spec	Feature creep, AI "helpful" thêm code	Vừa — dễ phát hiện
Regression Drift	Code xóa/thay đổi behavior đã define	Refactoring, performance optimization	Cao — breaks contracts
Terminology Drift	Tên khái niệm trong code ≠ trong spec	Team mới, naming convention changes	Vừa — gây confusion

Consistency Matrix

Consistency Gate không chỉ check code vs spec — nó check tất cả artifacts với nhau. Bất kỳ inconsistency nào ở bất kỳ layer nào đều là dấu hiệu của drift hoặc specification gap.

Cross-Artifact Consistency Matrix

	SPEC	PLAN	TASKS	CODE	TESTS
SPEC	—	✓	✓	✓	✓
PLAN	✓	—	✓	✓	-
TASKS	✓	✓	—	✓	-
CODE	✓	✓	✓	—	✓
TESTS	✓	-	-	✓	—

- SPEC vs CODE: Mọi SHALL trong SPEC có code implement
- SPEC vs TESTS: Mọi Acceptance Criteria có test, mọi test có spec ref
- PLAN vs CODE: Architectural decisions được reflect trong code structure
- CODE vs TESTS: Test coverage > threshold cho business logic

scripts/check_consistency.py

Automated Consistency Check — Python Script

```
import re, pathlib, sys
from dataclasses import dataclass, field

@dataclass
class ConsistencyReport:
    passed: list = field(default_factory=list)
    warnings: list = field(default_factory=list)
    failures: list = field(default_factory=list)

def check_spec_code_coverage(spec_dir):
    report = ConsistencyReport()
    spec_path = pathlib.Path(spec_dir)
    spec_text = (spec_path / "SPEC.md").read_text()

    # Check 1: Spec sections có task coverage
    spec_sections = re.findall(r"##\s+\d+\.\s+(.+)", spec_text)
    for section in spec_sections[:4]:
        tasks_text = (spec_path / "TASKS.md").read_text()
        if section.lower() not in tasks_text.lower():
            report.warnings.append(
                f"
```

Đồng bộ ngược từ Code về Spec

Sync-back là quá trình ngược lại của thông thường: code (đã thay đổi vì lý do hợp lệ) dẫn dắt việc cập nhật spec. Quan trọng: sync-back phải có chủ đích và documented.

Khi nào Sync-back là bắt buộc?

- Hotfix thay đổi error behavior khác spec → sync-back trong 24h
- Refactoring thay đổi data structure → sync-back trước merge
- Performance optimization thay đổi SLA → sync-back ngay
- Security patch thay đổi auth flow → sync-back + security review



Quy tắc vàng

Code change mà không có spec change = technical debt. Mọi thay đổi hành vi trong code đều phải có tương ứng trong spec.

AI-Assisted Sync-back

Prompt: Phát hiện và xử lý Drift

```
# Bước 1: Phát hiện drift
So sánh SPEC.md hiện tại với source code (thư mục src/).
Tìm và liệt kê TẤT CẢ sự khác biệt:

Output format:
| Type | Spec says | Code does | File:Line |
|-----|-----|-----|-----|
| Drift | rating: 1-5 | rating: 1-10 | svc:L45 |
| Code orphan | (not in spec) | export_to_pdf() | router:L89 |
| Spec orphan | bulk_import SHALL | (not implemented) | - |

# Bước 2: Quyết định cho từng item (Human review)
# A) Update spec để match code (code đúng, spec outdated)
# B) Update code để match spec (spec đúng, code drifted)
# C) Acknowledge và document (intentional deviation)

# Bước 3: AI update spec cho items type A
Cập nhật SPEC.md để phản ánh behavior thực tế của code.
Tạo SPEC_CHANGELOG entry với: ngày, item changed, lý do.
```

Bài tập — Consistency Gate

Bài tập 7.3.A — Drift Detection

- Cho một feature có SPEC.md, PLAN.md, TASKS.md và code đã deploy.
- Tìm tối thiểu 5 điểm drift giữa Spec và Code.
- Phân loại drift theo nhóm: missing behavior, extra behavior, semantic mismatch.
- Đề xuất thứ tự ưu tiên xử lý drift theo mức rủi ro.

Bài tập 7.3.B — Thiết kế Consistency Gate

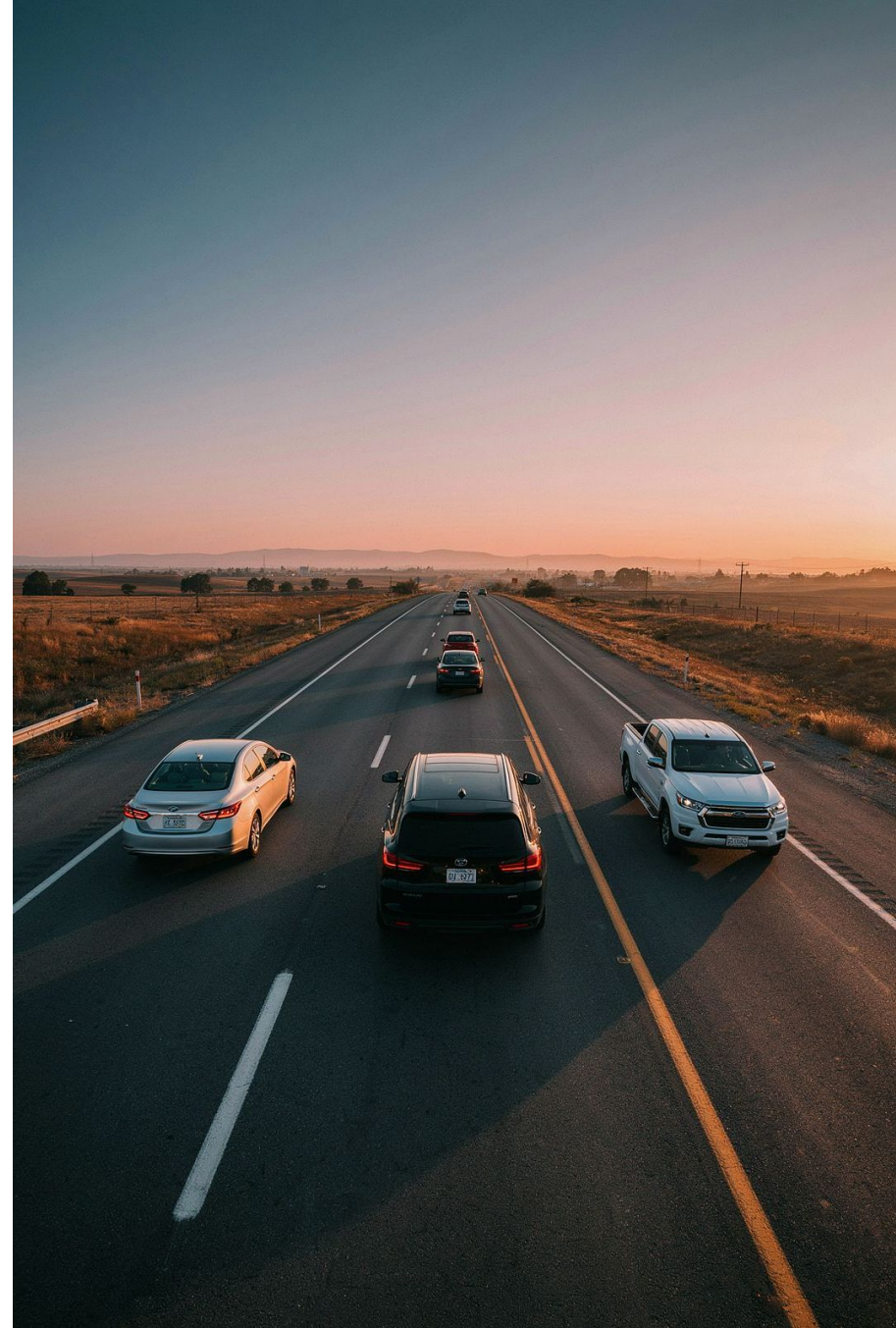
- Thiết kế 3 rules CI: EARS coverage, Out-of-Scope violation, Sync-back required.
- Đặt ngưỡng pass/fail rõ ràng cho từng rule.
- Viết mẫu output khi gate fail để dev biết phải sửa gì.
- Đề xuất cách giảm false positive trong dự án lớn.

Mục tiêu Biến consistency check thành quality gate vận hành được qua nhiều sprint, không chỉ là kiểm tra thủ công một lần.

SECTION 7.4

Parallel Implementation Exploration

Đặt hàng AI tạo nhiều phương án từ cùng một spec, rồi đánh giá trade-offs một cách có hệ thống



Use Cases & Anti-patterns

Pattern này đặc biệt có giá trị cho các quyết định kiến trúc quan trọng — những quyết định mà cost of change cao sau khi đã implement. Đầu tư 30 phút để AI tạo 2-3 phương án thường tiết kiệm nhiều ngày refactoring.

Tình huống	Dùng Pattern?	Lý do
Caching strategy (Redis vs local)	✔ Nên dùng	Trade-off phức tạp, cost of change cao
Database schema cho core entity	✔ Nên dùng	Schema migration rất tốn kém
API design (REST vs GraphQL)	✔ Nên dùng	Breaking change nếu đổi sau này
Queue technology (Kafka vs Celery)	✔ Nên dùng	Infrastructure commitment lớn
Implement utility function đơn giản	✘ Không cần	Overengineering, cost > benefit
Fix bug nhỏ, rõ ràng	✘ Không cần	Không có ambiguity về approach
CRUD boilerplate	✘ Không cần	Best practice đã clear

Parallel Exploration Prompt

PARALLEL EXPLORATION PROMPT — Template chuẩn

Đọc SPEC.md (đặc biệt phần Non-functional Requirements).
Tạo ĐÚNG 3 phương án implementation khác nhau cho
[mô tả vấn đề: ví dụ "caching cho product catalog"].

Với MỖI phương án, cung cấp:

Phương án [N]: [Tên ngắn gọn]

Mô tả kỹ thuật (3–5 câu)
[Giải thích cơ chế hoạt động]

Code skeleton (chỉ structure, không implement đầy đủ)

Trade-off Analysis

Tiêu chí	Điểm (1–5)	Nhận xét
-----	-----	-----
Latency	X/5	[chi tiết]
Cost	X/5	[chi tiết]
Complexity	X/5	[chi tiết]
Scalability	X/5	[chi tiết]
Maintainability	X/5	[chi tiết]

Best fit when / Worst fit when

So sánh tổng hợp + Khuyến nghị

KHÔNG implement chi tiết. Chỉ tạo phác thảo.
Mục tiêu: đủ thông tin để human ra quyết định.

Phương án 1 & 2: Local vs Redis Cache

Phương án 1 — Local In-Memory Cache

```
@lru_cache(maxsize=1000)
def get_product(id):
    return db.query(Product)\
        .filter_by(id=id).first()
```

- Latency: 5/5 — Microseconds
- Cost: 5/5 — Free
- Complexity: 5/5 — 1 decorator
- Scalability: 1/5 — Không scale

⚠ Cache không share giữa multiple instances!

Phương án 2 — Redis Distributed Cache

```
def get_product(id):
    cached = redis.get(f"product:{id}")
    if cached:
        return json.loads(cached)
    p = db.query(Product)\
        .filter_by(id=id).first()
    redis.setex(f"product:{id}", 300,
               json.dumps(p))
    return p
```

- Latency: 4/5 — ~1ms LAN
- Cost: 3/5 — ~\$20-50/month
- Complexity: 3/5 — Redis config
- Scalability: 5/5 — Tốt, shared

Phương án 3 & Khuyến nghị

Phương án 3 — CDN / Edge Caching

HTTP caching headers (Cache-Control) + CDN (Cloudflare/AWS CloudFront). Không cache trong application — delegate cho infrastructure.

- Latency: 5/5 — CDN edge: <10ms globally
- Scalability: 5/5 — Infinitely scalable
- Cache invalidation: 2/5 — CDN purge có latency, inconsistency

Best fit: Public catalog, không cần auth, global users.

Khuyến nghị: Phương án 2 (Redis)

Dựa trên SPEC.md Non-functional: "< 200ms (p95), tối đa 1000 categories" và hệ thống đang có multiple replicas trên Kubernetes. Redis đáp ứng scalability requirement. CDN bị loại vì ProductCatalog có auth requirement.

Chọn phương án với tiêu chí rõ ràng

Framework cân nhắc 5 chiều theo weight phù hợp với context của dự án — không phải "cái gì quen" hay "cái gì nghe có vẻ hay":

Tiêu chí	Trọng số	Khi nào weight cao	Khi nào weight thấp
Latency	1–3×	Real-time features, UX-critical	Batch jobs, background tasks
Cost	1–3×	Startup, cost-sensitive	Enterprise, well-funded
Complexity	1–2×	Small team, nhân lực khan hiếm	Large team, chuyên sâu
Scalability	1–3×	High-growth product	Internal tool, stable load
Maintainability	1–2×	Core system, long lifecycle	Prototype, throwaway code

Weighted Evaluation Example

Decision Scorecard — Chọn caching strategy cho ProductCatalog

```
# Project: SaaS product, multiple tenants, 50k users
weights = {
  "latency": 2, # Vừa — 200ms OK theo spec
  "cost": 1, # Thấp — budget không tight
  "complexity": 2, # Vừa — team có Redis experience
  "scalability": 3, # Cao — đang tăng trưởng nhanh
  "maintainability": 2, # Vừa — core product
}

options = {
  "local_cache": {"latency":5,"cost":5,"complexity":5,
  "scalability":1,"maintainability":2},
  "redis": {"latency":4,"cost":3,"complexity":3,
  "scalability":5,"maintainability":4},
  "cdn": {"latency":5,"cost":2,"complexity":5,
  "scalability":5,"maintainability":3},
}

# Kết quả:
# local_cache:  $1 \times 5 + 2 \times 5 + 2 \times 5 + 3 \times 1 + 2 \times 2 = 32/50 = 64\%$ 
# redis:  $1 \times 4 + 2 \times 3 + 2 \times 3 + 3 \times 5 + 2 \times 4 = 39/50 = 78\% \leftarrow \text{WINNER}$ 
# cdn:  $1 \times 5 + 2 \times 2 + 2 \times 5 + 3 \times 5 + 2 \times 3 = 40/50 = 80\%$ 

# CDN slightly higher BUT:
# CDN chỉ phù hợp public APIs — ProductCatalog có auth  $\Rightarrow$  Disqualify CDN
#  $\rightarrow$  CHỌN: Redis (78%, phù hợp context nhất)
```

Bài tập — Parallel Exploration

Bài tập 7.4.A — ★★

Lấy một technical decision bạn cần đưa ra trong project hiện tại (database design, queue technology, auth approach...). Dùng Parallel Exploration Prompt để AI tạo 3 phương án. Áp dụng Decision Scorecard để chọn. So sánh kết quả của framework với lựa chọn ban đầu của bạn.

Bài tập 7.4.B — ★★

Sau khi chọn phương án, viết một ADR (Architecture Decision Record) ngắn gọn trong `.sdd/rfcs/ADR- $\{n\}$ .md` ghi lại: context, options considered, decision, rationale, và consequences. ADR này sẽ là "spec as code" cho quyết định kiến trúc.



SECTION 7.5

Specification Scale Management

Kiến trúc Spec Hierarchy có cùng nguyên lý scaling như kiến trúc phần mềm: chia nhỏ, phân cấp, loose coupling

Ba Lớp Kiến trúc



Quy tắc không thể thương lượng

Global Spec chứa những quy tắc áp dụng cho toàn bộ hệ thống, bất kể service hay feature. Đây là nơi Constitution sống, cùng với các tài liệu định nghĩa "cách hệ thống hoạt động như một tổng thể":

File	Nội dung	Ví dụ rules
constitution.md	Hard/Arch/Eng rules	No secrets, soft-delete, auth required
system-architecture.md	High-level service topology, communication patterns	Event-driven, REST APIs, Kafka cho async
security.md	Security policies, compliance requirements	HTTPS only, OWASP top 10, GDPR
data-governance.md	Data ownership, retention, privacy rules	PII masking, 90-day log retention
api-standards.md	Cross-service API design conventions	REST naming, pagination, error format

Hợp đồng giữa các services

.sdd/specs/services/order-service/module.md

Owner: @order-team | Version: 2.3.0

Service Responsibility

order-service là authority duy nhất cho:

- Order lifecycle management (create → fulfill → close)
- Order state transitions
- Order item management (add/remove/update)

NOT trong scope của service này:

- Payment processing (→ payment-service)
- Inventory management (→ inventory-service)
- Shipping tracking (→ logistics-service)

API Contract (summary)

Base URL: /api/v2/orders

Auth: JWT Bearer (valid scope: orders:read, orders:write)

Rate limit: 1000 req/min per tenant

Internal Architecture

Pattern: Hexagonal Architecture (ports & adapters)

DB: PostgreSQL 16, schema: orders_db

Events published: order.created, order.fulfilled, order.cancelled

Events consumed: payment.confirmed, inventory.reserved

Module-level Constraints

- Mọi state transition phải publish event vào Kafka
- Không có synchronous calls đến payment-service
- Order history không được xóa (immutable audit log)

Sprint-level Artifact với Layer Inheritance

Mọi Feature SPEC.md phải explicitly reference layer cha nó thuộc về và phải inherit constraints từ đó. Chỉ list những rule THÊM VÀO, không lặp lại những gì đã có ở Layer 1/2:

.sdd/features/feat-bulk-order/SPEC.md

```
# INHERITS FROM:
# Layer 1: .sdd/specs/global/constitution.md
# Layer 2: .sdd/specs/services/order-service/module.md
# These rules apply automatically — not repeated here.

## 1. Context & Goal
B2B customers cần đặt nhiều orders cùng lúc.

## 2. Additional Constraints (feature-specific only)
# Chỉ list những rule THÊM VÀO, không có trong Layer 1/2
- Bulk order: tối đa 100 orders per request
- Response: return all-or-nothing
  (nếu 1 fail → rollback tất cả)

## 3-8. [Standard sections...]
```

Feature specs ngắn gọn hơn, không lặp lại.

Thay đổi ở Layer 1/2 tự động áp dụng cho mọi feature.

Single source of truth cho mọi constraint.

Git Strategy cho Spec

Repository structure — OPTION A: Spec trong cùng repo (monorepo)

```
my-monorepo/
├── .sdd/
│   ├── global/ # Layer 1 — shared across all services
│   │   ├── constitution.md
│   │   └── system-arch.md
│   └── services/
│       ├── order-service/
│       │   ├── module.md
│       │   └── features/
│       └── feat-bulk-order/
│       └── payment-service/
└── services/
    ├── order-service/ # Code
    └── payment-service/
```

Git tag strategy:

Format: spec/{layer}/{name}/v{semver}

git tag spec/global/constitution/v2.0.0

git tag spec/service/order-service/module/v1.5.0

git tag spec/feature/bulk-order/v1.0.0

Code PR description phải reference spec tag:

"Implements spec/feature/bulk-order/v1.0.0"

→ Traceability: từ any code commit → spec version tại thời điểm đó

Chọn chiến lược phù hợp

Chiến lược	Phù hợp với	Điểm mạnh	Điểm yếu	Tooling
Spec trong code repo (monorepo)	Startup, SME, single-team	Atomic commit code+spec, đơn giản	Spec dễ bị bỏ qua, không có enforcement riêng	GitHub Actions + .sdd/ folder
Spec trong code repo (multi-repo)	Growing startups, 2-5 services	Context rõ ràng per service	Specs không share được dễ dàng	Per-repo GitHub Actions
Separate spec repo	Enterprise, 10+ services	Global governance, single source	Submodule complexity, out-of-sync risk	Dedicated spec CI/CD pipeline
Spec in wiki (Confluence/Notion)	Traditional teams mới chuyển đổi	Quen thuộc, dễ edit	Không version control, không automation	Manual only

Đo lường và trả nợ Spec Debt

Spec Debt là spec đang thiếu, outdated, hoặc không đủ chất lượng. Đo lường giúp team biết khi nào cần đầu tư vào "spec refactoring":

scripts/spec_debt_report.py

```
def calculate_spec_debt(sdd_dir):
    debt = {
        "outdated_specs": [], # Specs cũ > 90 ngày không update
        "incomplete_specs": [], # Specs thiếu sections
        "orphan_specs": [], # Specs cho features đã deprecated
    }

    # Check 1: Specs quá cũ (> 90 ngày)
    for spec_file in sdd_path.rglob("SPEC.md"):
        age_days = (datetime.now().timestamp() -
                    spec_file.stat().st_mtime) / 86400
        if age_days > 90:
            debt["outdated_specs"].append({
                "file": str(spec_file),
                "age_days": int(age_days)
            })

    # Check 2: Specs thiếu required sections
    required = ["Context", "Actors", "Functional", "Out of Scope"]
    for spec_file in sdd_path.rglob("SPEC.md"):
        missing = [s for s in required
                    if s not in spec_file.read_text()]
        if missing:
            debt["incomplete_specs"].append({
                "file": str(spec_file),
                "missing": missing
            })

    # Tính Spec Debt Score (0-100, thấp = tốt)
    total = len(list(sdd_path.rglob("SPEC.md")))
    debt_items = (len(debt["outdated_specs"]) +
                  len(debt["incomplete_specs"]) * 2)
    score = min(100, int(debt_items * 100 / total))
    return {"spec_debt_score": score, "details": debt}
```

SECTION 7.6

Case Study — SDD cho dự án E-commerce (Greenfield)

SDD cho dự án E-commerce Greenfield — minh họa tất cả patterns đã học trong thực tế



EcoShop — B2C E-commerce Việt Nam

3

Developers

6

Sprints MVP

2

Core Modules

Tech Stack

- FastAPI — Backend API
- React — Frontend
- PostgreSQL — Primary DB
- Redis — Cache
- Kafka — Event streaming

Greenfield SDD Setup

- Khởi tạo Spec Hierarchy
- Tạo Global Constitution
- Module Specs cho Cart và Payment
- Feature Specs cho Sprint 1

Constitution — PDPA & Business Rules

.sdd/global/constitution.md — EcoShop (excerpt)

SEC-01: PII Protection (PDPA Vietnam compliance)

THE system SHALL mask customer phone và email trong logs:

phone: "0912***456", email: "user***@domain.com"

Violation = automatic block, security review required.

SEC-02: Payment data isolation

Payment card data SHALL NEVER touch application servers.

Card processing: delegated entirely to payment gateway

(VNPAY/Momo).

BUS-01: Giá hiển thị = Giá áp dụng

Giá trong order confirmation SHALL bằng giá đã hiển thị khi checkout.

Nếu giá thay đổi sau khi add to cart → user được thông báo.

BUS-02: Inventory optimistic locking

Stock reservation SHALL dùng optimistic locking.

Oversell prevention là bắt buộc cho mọi order path.

ARCH-01: Cart → Payment là async

Cart checkout trigger payment intent QUA Kafka event.

Không có synchronous HTTP call từ cart-service đến payment-service.

CONTEXT.md — Trước khi viết Spec

.sdd/services/cart-service/CONTEXT.md

Problem Statement

Pain points: (1) mất cart khi logout, (2) giá thay đổi không được báo, (3) không merge cart giữa devices.

Domain Knowledge

"Cart" = collection of CartItems.

CartItem: product_id, variant_id (optional), quantity, snapshot_price.

"snapshot_price" = giá tại thời điểm add (không phải giá live).

Open Questions (cần resolve trước Spec)

Q1: Guest cart: persist bao lâu?

Q2: Cart merge: khi guest login, user cart + guest cart

→ merge hay replace?

Q3: Max items: giới hạn không?

ANSWERS (sau product discussion)

A1: Guest cart: 7 ngày (localStorage + server-side session)

A2: Merge: add guest items vào user cart, resolve conflicts

bằng "keep higher qty"

A3: Max 50 items per cart (performance + UX)

Pattern nhận xét

Bước CONTEXT.md (Pha 0) đặt câu hỏi sản phẩm trước khi viết EARS requirements. Nếu không làm bước này, Clarification Trigger Prompt ở Pha 1 sẽ tốn nhiều round-trip hơn.

EARS Requirements — Cart Core

.sdd/features/feat-cart-core/SPEC.md — Functional Requirements

3. Functional Requirements (EARS)

WHEN Customer/Guest adds product, THE system SHALL:

- Check inventory: stock $\geq$ requested quantity
- Capture snapshot_price = current_price
- If product already in cart: increase quantity
- If new product: add CartItem
- Return updated cart state

WHEN Customer logs in with existing Guest cart,
THE system SHALL merge carts:

- For same product: keep $\max(\text{guest_qty}, \text{user_qty})$
- For guest-only items: add to user cart
- Delete guest cart after merge
- Refresh snapshot_price sau merge (từ AI Clarification Q1)

WHILE cart has items with outdated prices (> 15 min),
THE system SHALL refresh snapshot_prices on cart view
and flag changed items: price_changed: true.

4. Non-functional

- Cart view response: $< 100\text{ms}$ (p95)
- Cart operations: $< 200\text{ms}$ (p95)
- Guest cart TTL: 7 days

8. Out of Scope

- Không có coupon/promo code (Sprint 2)
- Không có saved-for-later / wishlist
- Không có cart sharing via link

Error Handling & Acceptance Criteria

Section 6: Error Handling (EARS Unwanted Patterns)

WHERE product.stock < requested_quantity,
THE system SHALL return 422 with:
{error: "insufficient_stock",
available: N, requested: M}

WHERE product is_active = false,
THE system SHALL remove from cart automatically
and return 200 with notice:
"unavailable_items_removed": [product_ids]

WHERE cart has > 50 items,
THE system SHALL return 422:
{error: "cart_limit_exceeded", max: 50}

WHERE cart merge creates > 50 items,
THE system SHALL keep top 50 by added_at desc
and notify: "Some items removed due to cart limit."

Section 7: Acceptance Criteria

- Add in-stock product → CartItem created, snapshot_price = current
- Add out-of-stock product → 422 insufficient_stock
- Add same product twice → quantity increased, not duplicate
- Login with guest cart → merged correctly (max qty rule)
- Cart view with stale prices → refreshed, changed flagged
- Add 51st item → 422 cart_limit_exceeded

3 Edge Cases AI phát hiện

Clarification Output — Cart Spec

AI Clarification Questions — Cart Spec

1. MERGE CONFLICT với `snapshot_price`:

Khi merge, CartItem được giữ với `snapshot_price` nào?

→ **ANSWER:** Recalculate fresh `snapshot_price` khi merge.

→ **SPEC UPDATED:** merge rule thêm "refresh `snapshot_price` sau merge"

2. CONCURRENT ADD: Nếu 2 tabs add cùng sản phẩm đồng thời?

→ **ANSWER:** DB upsert với quantity += requested (not replace)

→ **SPEC UPDATED:** "concurrent safe: upsert với increment"

3. VARIANT logic: Product + variant khác nhau = same CartItem hay different?

Ví dụ: Áo size M và Áo size L = 2 items hay 1?

→ **ANSWER:** Different items

(`product_id` + `variant_id` = unique key)

→ SPEC was correct, just implicit. Made explicit.

Post-clarification: SPEC.md tăng từ 45 → 58 lines

3 rules được thêm, không có rule nào bị bỏ

Components + Risks

Approach: Repository + Service pattern. Redis cache cho cart reads (TTL=60s). PostgreSQL cho persistence (source of truth).

Component	File	Responsibility
CartRepository	cart/repository.py	DB read/write (PostgreSQL)
CartCacheLayer	cart/cache.py	Redis read-through cache
CartService	cart/service.py	All business rules
PriceRefresher	cart/price_refresher.py	Stale price detection/update
CartRouter	cart/router.py	HTTP endpoints
CartMergeService	cart/merge_service.py	Login merge logic

Risk	Prob	Impact	Mitigation
Race condition in merge	High	High	DB transaction + lock
Price drift during add	Med	Med	Snapshot at add time
Cache invalidation miss	Med	Low	TTL + write-through

7.6.3 — PRODUCTION CODE

CartService với EARS Tags

cart/service.py — EARS tags traceability

```
class CartService:
    async def add_item(self, cart_id, product_id,
variant_id, quantity):
    # EARS[Event]: WHEN Customer/Guest adds product
    # EARS[Unwanted]: WHERE cart has > 50 items
    cart = await self.repo.get(cart_id)
    if len(cart.items) >= 50:
        raise CartLimitError(max=50)

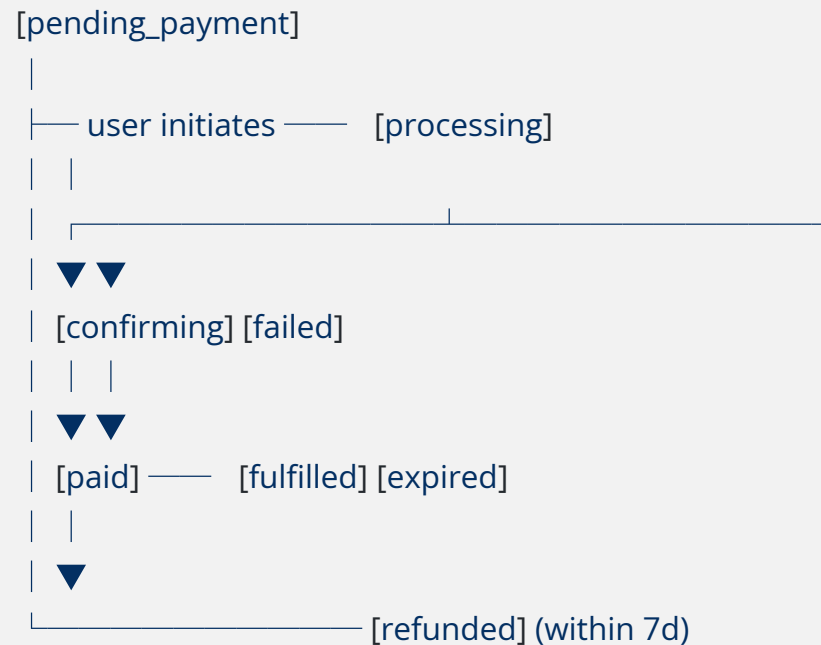
    # EARS[Unwanted]: WHERE product.stock < requested
    stock = await self.price_svc.get_stock(
product_id, variant_id)
    if stock < quantity:
        raise InsufficientStockError(
available=stock, requested=quantity)

    # EARS[Unwanted]: WHERE product is_active = false
    product = await self.price_svc.get_product(product_id)
    if not product.is_active:
        raise ProductUnavailableError(product_id=product_id)

    # EARS[Ubiquitous]: capture snapshot_price
    snapshot_price = product.current_price

    # EARS[Ubiquitous]: concurrent safe upsert
    # (clarification Q2)
    item = await self.repo.upsert_item(
cart_id=cart_id,
product_id=product_id,
variant_id=variant_id,
quantity_delta=quantity, # += not replace
snapshot_price=snapshot_price,
)
    await self.cache.invalidate(cart_id)
    return await self.repo.get(cart_id)
```

Formal Spec — State Machine



VALID TRANSITIONS:

`pending_payment` → `processing` : user submits payment
`processing` → `confirming` : gateway responds pending
`processing` → `failed` : gateway responds failure
`confirming` → `paid` : webhook payment.success
`confirming` → `failed` : webhook payment.failed
`failed` → `pending_payment` : user retries (within 30 min)
`failed` → `expired` : > 30 min without retry
`paid` → `fulfilled` : warehouse ships order
`paid` → `refunded` : admin initiates (within 7 days)

INVARIANTS:

- Chỉ có 1 `payment_attempt` active per order
- Mọi state transition có `audit_log` entry
- Amount KHÔNG thay đổi sau `pending_payment`

State Machine từ Spec

payment/state_machine.py — Generated từ SPEC.md

```
from transitions import Machine
from enum import Enum


class PaymentState(str, Enum):
    PENDING = "pending_payment"
    PROCESSING = "processing"
    CONFIRMING = "confirming"
    PAID = "paid"
    FAILED = "failed"
    EXPIRED = "expired"
    FULFILLED = "fulfilled"
    REFUNDED = "refunded"


# EXACTLY mirrors valid transitions trong SPEC.md
# Adding a transition here WITHOUT updating spec
# = Constitution violation
VALID_TRANSITIONS = [
    {"trigger": "initiate",
     "source": "pending_payment", "dest": "processing"},
    {"trigger": "gateway_pending",
     "source": "processing", "dest": "confirming"},
    {"trigger": "gateway_fail",
     "source": "processing", "dest": "failed"},
    {"trigger": "webhook_success",
     "source": "confirming", "dest": "paid"},
    {"trigger": "webhook_fail",
     "source": "confirming", "dest": "failed"},
    {"trigger": "retry",
     "source": "failed", "dest": "processing",
     "conditions": "within_retry_window"},
    {"trigger": "fulfill",
     "source": "paid", "dest": "fulfilled"},
    {"trigger": "refund",
     "source": ["paid", "fulfilled"], "dest": "refunded",
     "conditions": "within_refund_window"},
]


class PaymentOrder:
    def log_transition(self):
        # EARS[Ubiquitous]: every transition has audit_log
        AuditLog.create(order_id=self.id,
                        from_state=self.state,
                        to_state=self.dest)


# → Invalid transition attempt raises MachineError automatically
# → Spec is enforced at runtime, not just documentation
```

SDD cho Greenfield — Bài học thực tế

Lesson	Xảy ra khi nào	Giải pháp SDD
"Spec drift ngay trong Sprint 1"	Dev thêm price_history mà không update spec	Pre-commit check EARS tag + sync-back reminder
"AI tạo code oversell sản phẩm"	Constitution BUS-02 không rõ cơ chế	Thêm explicit optimistic locking rule vào Constitution
"Parallel Exploration tiết kiệm 2 ngày"	Team tranh luận Redis vs PostgreSQL cache	Decision Scorecard cho ra kết quả trong 1 tiếng
"Clarification tìm 4 edge cases"	Payment state machine spec review	AI hỏi concurrent payment initiation scenario
"Module Spec ngăn bad API design"	Dev muốn cart gọi sync API đến payment	ARCH-01 trong cart-service module.md block ngay

Tổng kết Chương 7

 Key Insight — "Spec as Code"

Điểm mạnh nhất của Spec as Code không phải là automation — mà là traceability.

Mọi dòng code đều có thể traced về spec. Mọi spec có thể traced về business decision.

Khi có bug: *"Spec nói gì?"* Khi có thay đổi business: *"Spec nào bị ảnh hưởng?"*

Pattern	Vấn đề giải quyết	Công cụ chính
7.1 Constitution-Driven Dev	Rules bị bỏ qua, AI tự tiện	constitution.md + CI/CD gates
7.2 Clarification-First	AI assume sai, code sai requirement	Clarification Trigger Prompt
7.3 Consistency Gate	Spec-Code drift tích lũy theo thời gian	check_consistency.py + Sync-back
7.4 Parallel Exploration	Chọn implementation sai vì thiếu comparison	Exploration prompt + Decision Scorecard
7.5 Scale Management	Spec chaos khi project lớn	Spec Hierarchy + Git strategy
7.6 Case Study	Greenfield không có SDD foundation	Full pipeline PRD → production code